

Dokumentation

Autonome Mobile Systeme

Projektabschlussarbeit

„Aufmerksamer Roboter“

Fach:	Autonome Mobile Systeme
Semester:	5. Semester
Studiengang:	Bachelor of Science Informatik
Teammitglieder:	Tobias Madaj (20200958) Fabian Claus (20130004)
E-Mail:	tobiasm@thbrandenburg.de claus@th-brandenburg.de

Inhaltsverzeichnis:

1. Analyse der Aufgabenstellung
2. Anforderungen & Randbedingungen
3. Diskussion von Lösungsvarianten
4. Konzeption und Lösungsideen
5. Implementierung
6. Einschätzung der Lösung & aufgetretene Probleme
7. Zusammenfassung
8. Vollständig kommentierter Quelltext
9. Literaturverzeichnis

1. Analyse der Aufgabenstellung:

Es soll ein Software-Programm entwickelt werden. Der Roboter (Sunny) soll ein Objekt, bzw. einen roten Ball zuverlässig im Zentrum des Bildes behaltet, indem er die Kamera bewegt und selbst sich zum Objekt rotiert. Die Maschine soll aber nicht dem Objekt hinterherfahren, sondern ihm aufmerksam folgen und sich immer wenn sich das Objekt bewegt mit rotieren. Das Objekt kann man am Anfang selbst definieren, es ist freigestellt und nicht fest vorgeschrieben im Projekt. Der Roboter soll das zu erkennende Objekt im Zentrum des Bildes behalten.

2. Anforderungen & Randbedingungen:

Die Teams müssen im voraus aus 2 Mitglieder bestehen. Es muss immer ständige und informative Ausgaben geben und das Team muss unter einen Zeitaufwand ihr können und Robustheit unter Beweis stellen. Es sollen elegante und weiche Bewegungen mit dem Roboter programmiert und ausgeführt werden, was zu schnelle Bewegungen komplett ausschließt. Jede Gruppe muss ihre eigene Lösung entwickeln. Der Bildverarbeitungsserver kann von Hand gestartet werden, sollte aber keine weiteren Einstellungen bis auf die Videoquelle haben.

Semester interne Kooperationen sind sinnvoll, aber jede Gruppe muss ihre eigene Lösung entwickeln und es darf nicht betrogen werden. Des weiteren müssen die Roboter aus dem KI-Labor genutzt werden. Als übergeordnete Programmiersprache kommt im Projekt C++ zum Einsatz und Microsoft Visual Studio als übergeordnete Entwicklungsumgebung. Für die Bildverarbeitung wird die Bibliothek OpenCV benutzt, die in der Industrie großräumig benutzt wird und als Industriestandard angesehen werden kann. Es ist elementar das Objekt im Zentrum zu halten bei den Bewegungen.

3. Diskussion von Lösungsvarianten:

Erste Variante einer Bedürfnis befriedigenden Lösung ist es durch C++ die Kamera so zu programmieren dass sie ein Objekt erkannt wird, in dieser Variante ein Schachbrettmuster auf ein Papier gedruckt. Diese Lösungsvariante wurde versucht umzusetzen aber die Erkennung der vielen Schachbrettmuster verbraucht sehr viel Leistung wie haben nur 1 Bild pro Sekunde bekommen was viel zu wenig war um eine performante Anwendung zu programmieren. Im direkten Vergleich zur mit der zweiten Lösungsvariante haben wir nur ein Bruchteil der Performance bekommen. Die Verarbeitung des Schachbretts als Muster war zu Leistungshungrig und wir haben nun mal sehr wenig Rechenleistung auf dem Rechner des Roboters.

Zweite Variante einer Bedürfnis befriedigenden Lösung ist es durch C++ die Kamera so zu programmieren dass sie ein Objekt erkennen tut, in dieser Variante ist es ein Ball zu erkennen der einfacher zu erkennen ist und wir haben das 6 fache an Bildern pro Sekunde bekommen, was mit der ersten Variante nicht denkbar gewesen wäre. Die Verarbeitungsgeschwindigkeit ist wesentlich performanter bei der sehr schwachbrüstigen Hardware auf unseren Robotern.

4. Konzeption und Lösungsideen

Die Programmierung lässt sich grob in 2 Teile teilen. Zum einen die Bewegung des Roboters mithilfe von Aria und zum anderen die Bildverarbeitung durch OpenCV.

Bei der Bewegungssteuerung ergeben sich wiederum 2 Möglichkeiten:

- der Roboter übernimmt die komplette horizontale Rotation und die Kamera bewegt sich nur vertikal (Tilt)
- die Kamera verfolgt das Objekt soweit wie möglich und erst wenn dort die Grenzen erreicht wurden, bewegt sich auch der Roboter selbst

Die Geschwindigkeit der Rotation wird erhöht, je weiter sich das Objekt vom Zentrum des Bildes entfernt befindet.

Bei der Bildverarbeitung bieten sich nahezu unbegrenzt viele Möglichkeiten der Implementierung. Grundsätzlich müssen jedoch folgende Stufen durchschritten werden:

- Konvertierung des Kamerabildes in den geeignetsten Farbraum
- Thresholding des Bildes entsprechend der erwarteten Farbe
- Morphologische Operationen / Operationen zur Bereinigung des Bildes
- Erkennung der erwarteten Form
- Berechnung der Position des Objektes im Bild

5. Implementierung:

Die Grundlage für die Implementierung bildet das ActionBV-Beispiel. In diesem wurden die ActionBV.cpp und Thread.cpp angepasst. Bei der Implementierung wurde sich für die Variante entschieden, dass der Roboter die links / rechts Rotation übernimmt und die Kamera den Tilt.

Es wird überprüft ob sich die von der Bildverarbeitung gelieferte Position innerhalb des spezifizierten Bereiches um das Bildzentrum herum befindet.

Sollte dies der Fall sein bewegt sich weder der Roboter noch die Kamera. Liegt die Position außerhalb des Zentrums bewegen sich entweder die Kamera, der Roboter oder beides.

Für den Tilt-Wert der Kamera wird die y-Position auf das Intervall $[-2,2]$ gemappt. Hierdurch wird erreicht, dass sowohl eine Maximalgeschwindigkeit gesetzt wird, als auch, dass die Geschwindigkeit verringert wird, je näher das Objekt dem Zentrum des Bildes kommt.

Dieses Prinzip wird auch auf der horizontale Rotation des Roboters angewendet, mit dem Unterschied, dass die Geschwindigkeit nicht auf einen festen Bereich gemappt wird, sondern direkt mit dem x-Wert gerechnet wird.

Sollte das Objekt nicht sichtbar sein, wird die momentane Rotation (in die Richtung, in der das Objekt zuletzt gesehen wurde) fortgesetzt. Die horizontale Rotation wird hier jedoch auf einen festen Wert gesetzt.

Es werden außerdem während des ganzen Prozesses mehrere Wahrheitswerte (Bool-Werte) gesetzt, welche es ermöglichen am Ende informative Logs bezüglich der Position des Objektes auszugeben. Bei der Implementierung der Bildverarbeitung wird zunächst das von der Kamera kommende Bild in den HSV-Farbraum umgewandelt, um die Vorteile der Trennung des Farbwertes und der Helligkeit und Sättigung nutzen zu können.

Auf dieses wird die „inRange()“ Methode ausgeführt, welche ein Graustufenbild als Ergebnis liefert, welches an den Stellen weiß ist, wo der gewünschte Farbwert (bzw. Bereich, da Scalare als Parameter angegeben werden) ist und sonst Schwarz.

Auf diese Maske, werden die morphologischen Operationen Öffnung und Schließung sowie ein Blur angewendet um kleine Fragmente verschwinden zu lassen, Löcher in Objekten zu schließen und die Erkennung von Formen zu erleichtern.

Auf das resultierende Bild wird die HoughCircles-Kreiserkennung angewendet. Sollten Kreise gefunden werden, wird von allen die Fläche berechnet und der größte Kreis wird weiter verwendet. Sollte die Fläche dieses Kreises über einem Schwellwert liegen, wird er als das gewünschte Objekt erkannt. Der Schwellwert dient dazu, ggf. erkannte kleine Kreise im Hintergrund nicht zu erkennen, während das eigentliche Objekt zurzeit nicht sichtbar ist.

Wurde ein gültiges Objekt erkannt, wird dessen Position an die Robotersteuerung übergeben und der Kreis wird als Feedback an der entsprechenden Stelle auf das Kamerabild gezeichnet.

6. Einschätzung der Lösung & aufgetretene Probleme:

Probleme treten bei dieser Lösung durch den automatischen Helligkeitsausgleich der Kamera auf. Wenn die Kamera zu einer starken Lichtquelle zeigt, wird das dunkel genug, dass selbst die Konvertierung in den HSV-Farbraum keine zufriedenstellenden Ergebnisse liefert. An dieser Stelle würde man das Bild normalisieren. Da die Lichtverhältnisse sich aber je nach Position des Roboters ändern können müsste hier ein adaptiver Algorithmus genutzt werden. Die aktuelle Lösung kommt ca. auf 3 FPS, was gerade so an der unteren Grenze des noch Funktionierenden ist. Wenn zusätzlich ein adaptiver Algorithmus zur Normalisierung angewandt wird, sinkt der FPS-Wert in einen komplett unbrauchbaren Bereich.

Ein weiteres Problem stellt der Sync-Cycle des Roboters dar. Dieser sollte mit 100ms laufen und ab 250ms wird eine Warnung ausgegeben. Hier kam es ohne erkennbaren Zusammenhang zu anderen Faktoren oft zu Warnungen im 500-1000 ms Bereich. Durch die Bildverarbeitung ist der FPS-Wert sowieso sehr niedrig aber durch den langsamen Sync-Cycle kann es vorkommen, dass die Daten der wenigen Frames die verarbeitet werden nicht rechtzeitig von der Robotersteuerung weiterverarbeitet werden können und damit obwohl das Objekt korrekt erkannt wurde, der Roboter nicht darauf reagiert.

7. Zusammenfassung:

Das Projekt befasst sich mit einem Roboter, der aufmerksam auf ein bestimmtes Objekt wartet, daher der Name des Projektes „Aufmerksamer Roboter“. Wenn das entsprechende Objekt erscheint, wird entsprechend reagiert, der Roboter soll die Kamera in Richtung des Objektes drehen inklusiv seines eigenen Körpers, was eine aufmerksame Haltung widerspiegelt, jedoch soll er das Objekt nicht selbst verfolgen, sondern stehen bleiben. Es sollen keine Bewegungen stattfinden, die zu schnell sind, und jedes Team muss aus 2 Teammitgliedern bestehen. Das Objekt, was erkannt werden soll, ist freigestellt und obliegt der freien Wahl der Studenten. Das Team soll somit seine Robustheit und können beweisen.

Das vorliegende Kapitel befasst sich mit der Implementierung der Software, was die eigentliche Aufgabe war. Zuerst wird das ActionBV-Beispiel als Vorlage verwendet in Visual Studio, um eine Grundlage zu haben. Es wurden die Thread.cpp und ActionBV.cpp editiert in der Projektmappe. Bei der Implementierung wurde sich für die Variante entschieden, dass der Roboter die links / rechts Rotation übernimmt. Es wird überprüft, ob die von der Bildverarbeitung gelieferte Position innerhalb des Bildzentrums sich befindet. Ist das der Fall, bewegt sich weder der Roboter noch die Kamera. Liegt die Position außerhalb des Zentrums, bewegen sich entweder die Kamera, der Roboter oder beides.

Für den Tilt-Wert wird die y-Position auf ein Intervall gemappt. Das bewirkt, dass sowohl eine Maximalgeschwindigkeit existiert, als auch, dass die Geschwindigkeit sich verringert, je näher das Objekt dem Zentrum des Bildes kommt.

Dieses Prinzip wird auf die Rotation des Roboters angewendet, mit dem Unterschied, dass die Geschwindigkeit nicht auf einen festen Bereich gemappt wird, sondern direkt mit dem x-Wert gerechnet wird. Ist das Objekt nicht sichtbar, wird die Rotation fortgesetzt. Die horizontale Rotation wird hier jedoch auf einen festen Wert gesetzt. Bei der Bildverarbeitung wird das Bild in den HSV-Farbraum umgewandelt.

Dann wird die „inRange()“ Methode ausgeführt. Auf diese Maske, werden die morphologischen Operationen Öffnung und Schließung sowie ein Blur angewendet, um kleine Fragmente verschwinden zu lassen, Löcher in Objekten zu schließen und die Erkennung von Formen zu erleichtern. Darauf wird die HoughCircles-Kreiserkennung angewendet. Wenn Kreise gefunden werden, wird von allen die Fläche berechnet und der größte Kreis wird weiter verwendet. Sollte die Fläche dieses Kreises über einem Schwellwert liegen, wird er als das gewünschte Objekt erkannt. Wurde ein gültiges Objekt erkannt, wird dessen Position an die Robotersteuerung übergeben, an der entsprechenden Stelle auf das Kamerabild gezeichnet.

Zum Ende hin und als abschließende Worte lassen sich sagen, dass das Projekt „Aufmerksamer Roboter“ ein Projekt ist, das es auf den ersten Blick für viele als einfach erscheinen mag, aber man zum Abschluss hin eher auf Fehler trifft, die man lösen muss, und es nur so scheint, tut das es einfach wäre. Das größte Problem, dass die Bildverarbeitung zu langsam hat, die größte Zeit gedauert zu lösen. Zusammenfassend lässt sich sagen, dass das Projekt erfolgreich beendet wurde mit einer zufriedenstellenden Lösung, in dem sich der Roboter nur als aufmerksamer Beobachter verhält und die aufgeführten Probleme effektiv gelöst wurden.

8. Vollständig kommentierter Quelltext:

Nachfolgend kommen die relevantesten Teile des Quellcodes. Die kompletten Dateien befinden sich angehängt. Der Quelltext ist vollständig kommentiert wie gewünscht.

Thread.cpp:

```
while( !bEndThread ){

    //variables to save values of biggest circle
    int maxCircle = -1;
    double maxArea = 0;
    Point maxCenter;
    int maxRadius;

    Mat hsv, mask;
    vector<Vec3f> circles;

    // Holen aktuelles Bild
    pImage = pCamera->GetFrame();           // Zeiger auf CImage
    p = pImage->GetImage();                 // Zeiger auf IplImage
    // Konvertiere Bild-Metadaten von *IplImage zu cv::Mat, Bilddaten sind gemeinsam
    // Konvertierung erst in der Hauptschleife, da p durch das Grabben zwischen zwei Bildpuffern wechselt

    mat_p = cvarrToMat(p);

    // Auflösung des Bildes bestimmen
    xmax = pImage->Width();
    ymax = pImage->Height();

    cvtColor(mat_p,hsv, COLOR_BGR2HSV); //Convert the captured frame from BGR to HSV
    inRange(hsv, Scalar(iLowH, iLowS, iLowV), Scalar(iHighH, iHighS, iHighV), mask); //Threshold the image

    //morphological opening (removes small objects)
    erode(mask, mask, getStructuringElement(MORPH_ELLIPSE, Size(10, 10)));
    dilate(mask, mask, getStructuringElement(MORPH_ELLIPSE, Size(10, 10)));

    //morphological closing (removes small holes)
    dilate(mask, mask, getStructuringElement(MORPH_ELLIPSE, Size(5, 5)));
    erode(mask, mask, getStructuringElement(MORPH_ELLIPSE, Size(5, 5)));

    GaussianBlur(mask, mask, cv::Size(9, 9), 2, 2);

    //find circles in mask (greyscale image)
    HoughCircles(mask, circles, CV_HOUGH_GRADIENT, 1, mask.rows / 4, 200, 15, 0, 0);

    //check which of the detected circles is biggest
    for (uint i = 0; i < circles.size(); i++)
    {
        Point center(cvRound(circles[i][0]), cvRound(circles[i][1]));
        int radius = cvRound(circles[i][2]);
        double area = PI * radius * radius;
        if (area > maxArea){
            //current circle is bigger than the currently biggest one (or the first to be checked)
            maxArea = area;
            maxCircle = i;
            maxCenter = center;
            maxRadius = radius;
        }
    }

    cvtColor(mask, mat_ph[0], COLOR_GRAY2BGR);

    if (maxCircle != -1 && maxArea > 500){
        //we have found at least 1 circle
        //that circles area is > 500 to exclude small fragments
        //-> use it for tracking
        circle(mat_p, maxCenter, maxRadius, Scalar(0, 255, 0));
        shared_mem->SM_SetFloat(1, (float)maxCenter.x - xmax / 2);
        shared_mem->SM_SetFloat(2, (float)maxCenter.y - ymax / 2);
        shared_mem->SM_SetFloat(3, (float)1);
        Sleep(50);
    }
    else
    {
        //no circle found
        shared_mem->SM_SetFloat(3, (float)0);
    }

    // Das gewünschte Bild anzeigen
    if (iBild_Index == ANZ_BILD) pDlg->ShowImage( pImage );
    else pDlg->ShowImage( &(aImage[iBild_Index]));

    // Man muss auch loslassen koennen
    pCamera->ReleaseFrame( pImage ); // Frame freigeben

    dThreadFrameCount += 1;
}
```

ActionBV.cpp:

```
ArActionDesired *ActionBV::fire(ArActionDesired currentDesired)
{
    m_Desired.reset();
    //track general position for logs
    bool top;
    bool left;
    bool visible;
    bool center;

    if (m_Camera != NULL){
        center = false;

        //map the current y-Value to a range of [-2,2] for tilt speed of the camera
        float tiltMapping = output_start + m_Slope * (*m_Y - input_start);
        if (*m_Y > m_Border)
        {
            //object is in the top half of the image
            top = true;
            m_Camera->tiltRel(tiltMapping);
        }
        else if (*m_Y < m_Border * -1)
        {
            //object is in the bottom half of the image
            top = false;
            m_Camera->tiltRel(tiltMapping);
        }
        else
        {
            //object is inside specified center
            center = true;
        }
    }

    //prüfe ob der Schwellwert überschritten ist
    //und lege die Drehgeschwindigkeit, abhängig der Objektposition fest

    float* m_Visible = m_SharedMemory->SM_GetFloat(INDEX_VISIBLE);

    if (*m_Visible == 0){
        //object is not visible
        //rotate in the direction the object was last seen to look for it
        center = false;
        visible = false;
        if (*m_X < 0){
            //object was last seen in the left half of the image
            left = true;
            m_Desired.setRotVel(15);
        }
        else
        {
            //object was last seen in the right half of the image
            left = false;
            m_Desired.setRotVel(-15);
        }
    }
    else {
        //object is visible
        visible = true;
        if (*m_X > m_Border || *m_X < m_Border * -1)
        {
            //object is not inside the specified center
            center = false;
            //calculate speed based on x-Value
            float rotVel = *m_X / 5 * -1;

            //check on which side the object is
            if (rotVel > 0){
                left = true;
            }
            else
            {
                left = false;
            }
            m_Desired.setRotVel(rotVel);
        }
        else
        {
            //object is inside specified center
            center = true;
        }
    }

    if (center){
        ArLog::log(ArLog::Terse, "Object is near the center");
    }
    else
    {
        string vPosition = string((top ? "top" : "bottom"));
        string hPosition = string((left ? "left" : "right"));
        string vis1 = string(visible ? "visible" : "not visible");
        string vis2 = string(visible ? "is" : "was last seen");

        string msg = string("Object is ") + string(visible ? "visible" : "not visible") + string(" and ") +
            string(visible ? "is" : "was last seen") + string(" in the ") + string(top ? "top" : "bottom") + string(left ? " left" : " right");
        cout << msg << "\n";
    }

    return &m_Desired;
}
```


9. Literaturverzeichnis:

Blatt der Aufgabenstellung

Aria Dokumentation Reference HTML Seiten lokal auf dem Labor-Rechner

Folien auf Moodle

<https://moodle.th-brandenburg.de/>

OpenCV Dokumentation:

<https://docs.opencv.org/4.0.1/index.html>

Redewendungen für diese Arbeit die verwendet wurden:

https://webcache.googleusercontent.com/search?q=cache:NdhkTD66bGAJ:https://www.ph-freiburg.de/fileadmin/dateien/zentral/schreibzentrum/typo3content/Lehre_SS13/Redemittel_f%25C3%2583_r_schriftliche_wissenschaftliche_Texte.pdf+%26cd=1%26hl=de%26ct=clnk%26gl=de%26client=firefox-b-ab